LOCAL COLONY-MANAGEMENT SOFTWARE

# Two Portals, One Colony

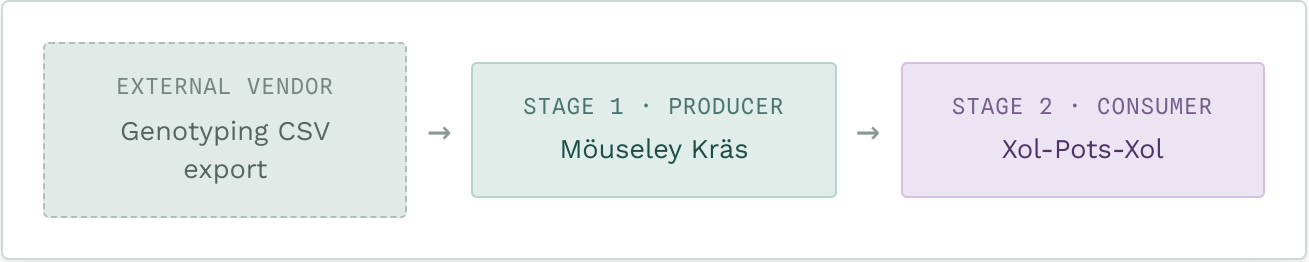
**Möuseley Kräs** turns a manually downloaded genotyping export into a reconciled mouse inventory, an audit trail, and printable cage cards. **Xol-Pots-Xol** takes the sparse cage-card workbooks that produces and consolidates them into fuller ones. Two small, separately-owned tools, one shared job: keep a mouse colony's paperwork honest.

```
3xtg_probe_09/HET/CALL_OK → K/+
```

## How the two fit together

SYSTEM RELATIONSHIP

The relationship is one-way. Möuseley Kräs is the only producer of cage-card workbooks in this system; Xol-Pots-Xol is a pure downstream consumer. It never reads Möuseley Kräs's inventory, its raw genotyping files, or its source code, and never writes back into Möuseley Kräs's template — it only ever reads finished workbooks and writes a brand-new one.



STAGE 1 — PRODUCER

# Möuseley Kräs

Genotyping → inventory → cage cards

Möuseley Kräs turns a manually downloaded genotyping-result export into three things: a reconciled copy of the mouse inventory, an audit and exception report explaining exactly what happened to every record it touched, and a cage-card workbook ready to print for the vivarium. A second, independent mode registers brand-new litters into the inventory before they've been genotyped at all. Its defining feature is **safety** — it is built to be trustworthy with irreplaceable colony records, even though it runs as an ordinary local program with no database and no operations team behind it.

## NINE IDEAS THE DESIGN LEANS ON

01 **Functional core, imperative shell**

The logic that decides what should happen — does a genotype match an approved pattern, do a litter's pup counts add up — is pure and untestable-to-break; the code that touches files, the network, or a subprocess is a thin shell around it.

02 **Configuration as data, not code**

Which column holds which field, what an "approved" genotype looks like, which cell on the cage-card template maps to which output — all declarative data in one config file, never hard-coded logic.

03 **Fuzzy-tolerant parsing, zero-tolerance matching**

Lenient about finding the right column when a header's casing or punctuation drifts; strict about matching a mouse to a record — always an exact key, never a guess.

04 **Fail-closed, explicit-outcome auditing**

Every record gets one status from a closed set — ready, conflict, needs review — with no silent "nothing to report" path. A conflicting genotype is preserved and reported, never overwritten.

---

05 **Checksum-verified, copy-on-write persistence**

Before touching the inventory, it hashes the existing file and takes a verified backup. Updates land on a new copy — never in place on the source.

---

06 **Idempotency via content hashing**

Every input file is hashed and checked against a small index before processing. Re-running the same export twice is caught and blocked by default.

---

07 **Process isolation across a language boundary**

Genotype translation runs in R; orchestration and the web layer run in Python. They talk over a subprocess with an explicit argument list — never a shell string.

---

08 **Two independent portals, one boundary**

Cage Card Production and Mouse Inventory Update share the same safety primitives but are otherwise separate code paths, deliberately kept from blurring into each other.

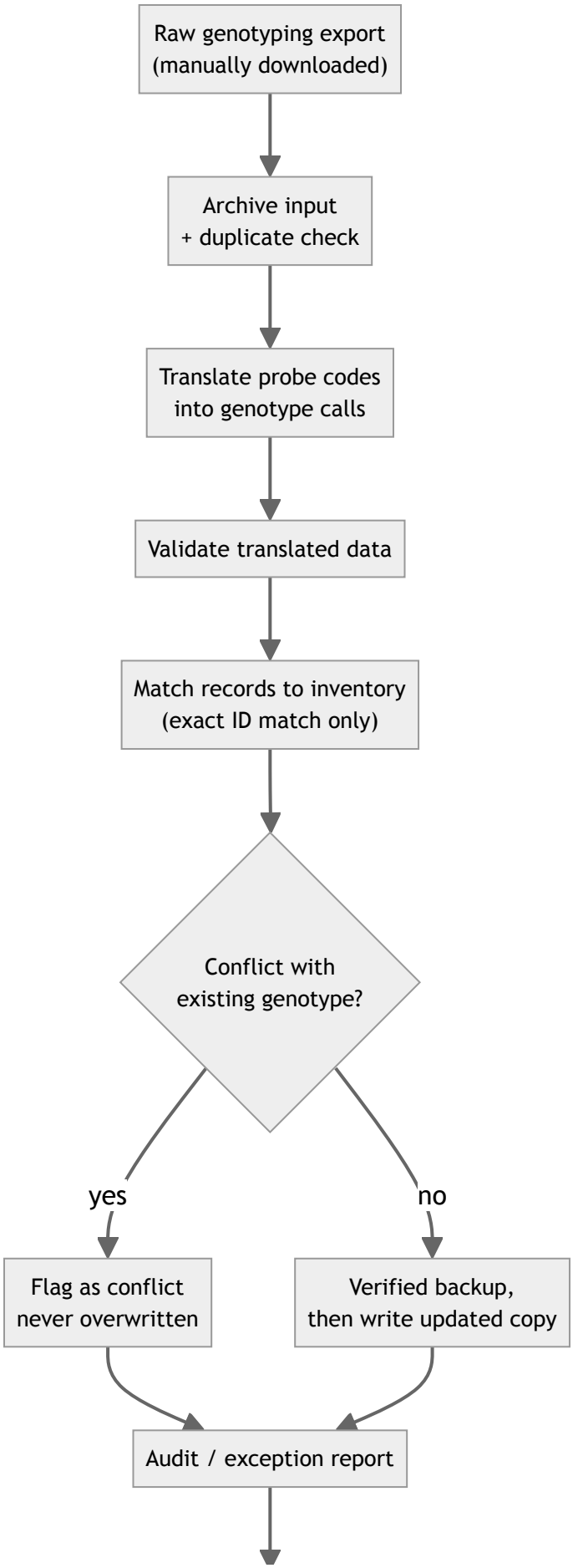
---

09 **Least-privilege external integration**

The one live external touchpoint — an optional overlay reading two date fields from a shared spreadsheet — is read-only, fills blanks only, and degrades to a warning rather than aborting a run if it can't be reached.

---

## CAGE CARD PRODUCTION, STEP BY STEP


Raw genotyping export
(manually downloaded)
Archive input
+ duplicate check
Translate probe codes
into genotype calls
Validate translated data
Match records to inventory
(exact ID match only)
Conflict with
existing genotype?
yes
no
Flag as conflict
never overwritten
Verified backup,
then write updated copy
Audit / exception report

Generate cage-card workbook

One run, one shared identifier, every record accounted for.

STAGE 2 — CONSUMER

# Xol-Pots-Xol

Sparse cage cards → one fuller workbook

Möuseley Kräs produces cage cards one small batch at a time, so a colony ends up with many *sparse* workbooks — each covering a handful of cages. Xol-Pots-Xol reads a set of those already-produced workbooks and merges compatible entries into fewer, fuller ones, without ever touching Möuseley Kräs's inventory, raw data, or template.

## SIX IDEAS THE DESIGN LEANS ON

01 **Narrow, explicit coupling**

Almost everything it reads is opaque text, copied straight through. The one exception — the genotype locus it needs to group by — lives in one small, named, versioned function, not a general parser trying to understand every possible format.

02 **Fail open to "don't merge," not to a guess**

A mouse the tool can't confidently classify goes into an explicit, reasoned-out "unconsolidated" bucket rather than a guessed grouping.

03 **Read, transform, write fresh**

It never edits a workbook in place. It reads several sparse ones, builds an in-memory model, and writes one brand-new output — trivially re-runnable, side-effect-free.

04 **Schema as contract**

Reads by fixed column position, but only after validating the header row — a stricter contract than Möuseley Kräs's, because its own inputs are far more uniform than a third-party export.

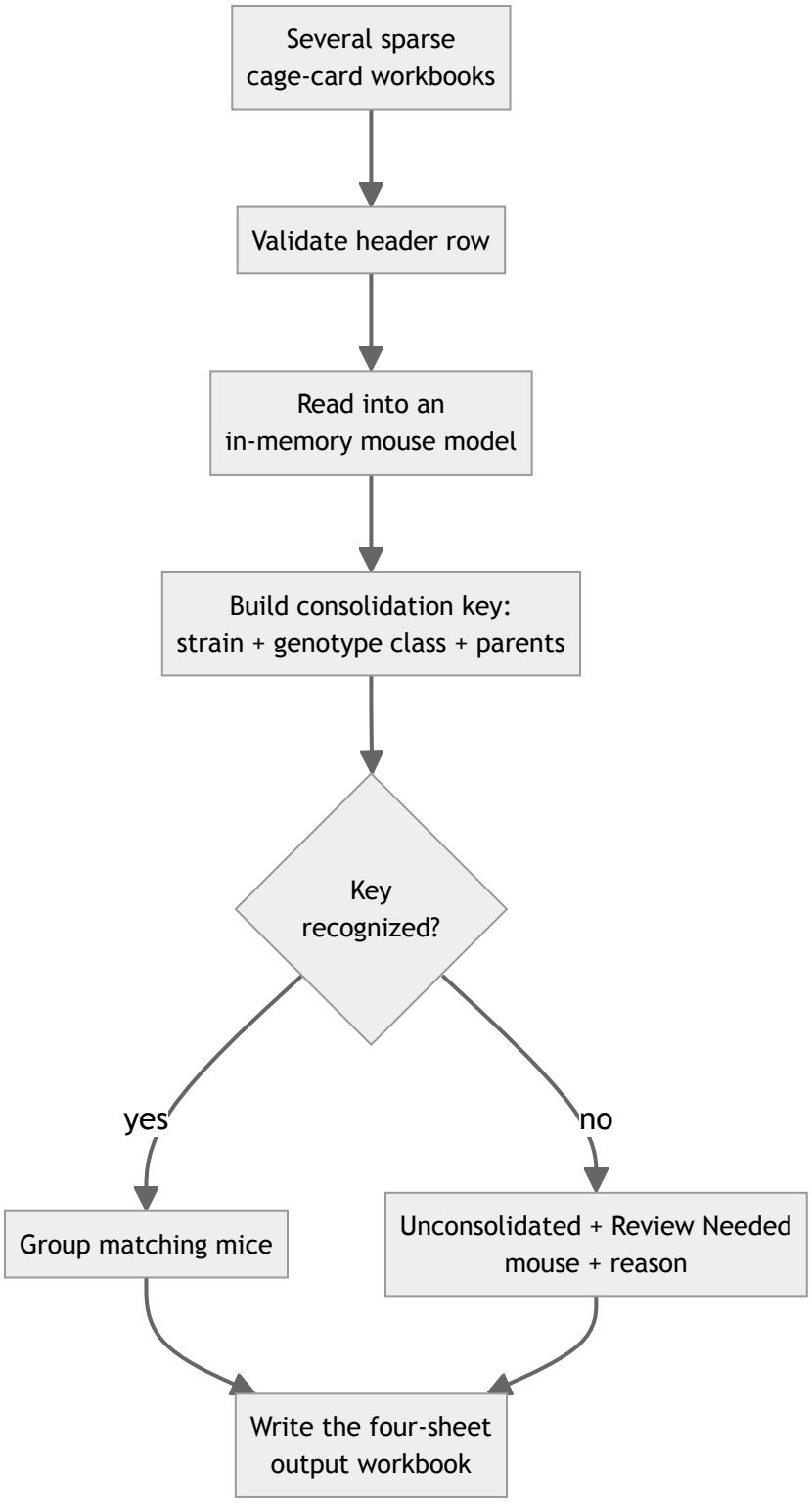
05 **A deterministic consolidation key**

Mice are grouped by strain, a normalized genotype-equivalence class, and parentage — two differently-worded genotype strings meaning the same thing collapse to one key.

06 **Separate "done" from "needs a human"**

The output always carries four sheets: consolidated cages, unconsolidated mice kept structurally apart, a review list with a named reason per mouse, and a reproducibility report.

CONSOLIDATION FLOW




Nothing is merged on a guess; everything unmerged says why.

## At a glance

COMPARISON

| ASPECT | MÖUSELEY KRÄS | XOL-POTS-XOL |
|---|---|---|
| Role | Producer | Consumer |
| Column matching | Lenient header resolution, strict identity match | Strict header contract, fixed column position |
| Domain coupling | Deep — genotype validation, inventory matching, cell mapping | Shallow, except one narrow, versioned locus rule |
| Persistence | Checksum-backed copy-on-write | Always a brand-new workbook; inputs never mutated |
| External integration | One opt-in, read-only, audited overlay | None |
| Uncertainty handling | Explicit per-record audit status | Explicit unconsolidated / review sheets with named reasons |
| Interfaces | Command line + local web app, two portals | Command line + local web app, one command |

## Where things stand

RUN SUMMARY

PORTABILITY INITIATIVE — UPDATED 2026-08-28

| | | |
|---|---|---|
| Phase 1 | Version control and scoping<br>Established as a real, staged goal | Done |
| Phase 2 | Isolated environments, locked dependencies<br>Verified across Python 3.11–3.14 | Done |
| Phase 3 | Windows and Linux launcher scripts<br>Implemented; Linux proxy-tested, Windows not yet executed | Partial |
| Phase 4 | Verification on real Windows/Linux machines or CI<br>Not started | Pending |

MACOS

**Verified**

The only platform any real run has executed on. Both suites pass in full.

LINUX

**Proxy-tested**

Launchers exist and were smoke-tested via a shell on macOS — a reasonable proxy, not a real machine.

WINDOWS

**Unexecuted**

Launchers are implemented and inspected, but have never run under a real interpreter.

This page describes the design and current status of two local, single-lab tools. It contains no animal records, genotype data, or credentials — it exists to explain how the software works and what it can currently be trusted to do.

Both tools run locally, one lab at a time, with no server infrastructure of their own.